

Tab PNF 論文まとめ

<https://arxiv.org/abs/2207.01848>

ABSTRACT

一言でまとめると

新しい表ごとに学習し直すのをやめ、事前に合成データで「例から規則を読み取る練習」を積んだ Transformer に、表をそのまま見せて答えさせる。この方法で、小さな表の分類を1秒未満で解ける。

できること：表形式のデータに対しての分類

順を追った整理

1. **何を作ったか**: TabPFN という学習済みの Transformer です。小さな表の分類を1秒未満で解き、ハイパーパラメータ調整は不要で、精度は最先端の手法に匹敵します。次のステップで、その解き方を説明します。
2. **使うときの解き方 (ICL) (推論方法)**: 正解つきの行 (例) と、正解が隠れた行 (問題) を、まとめて入力します。1回の計算 (順伝播) で、問題の行の答えが全部出ます。この間、重みは更新しません。次のステップは、この能力の作り方です。
3. **その能力の作り方 (PFN) (学習方法)**: 開発時に一度だけ、合成データの表で練習します。「一部の行の答えを隠し、残りの行から当てる」練習を、何億回も繰り返します。身につくのは、特定の規則ではなく、例から規則を読み取る手順です。その手順が重みの中に入っています (fully entailed in the weights)。次のステップは、練習用の表の作り方です。
4. **練習用の表の作り方 (prior と SCM)**: 原因から結果へのつながり (SCM) をランダムに作り、そこから表を生成します。表ごとにつながりは違います。単純なつながりが選ばれやすい設計で、これがオッカムの剃刀にあたる部分です。次のステップは、この方法の成果です。
5. **結果**: 条件つきで、次のとおりです。
 - ・ 訓練データ 1,000 件以下、数値の特徴のみで 100 個以下、欠損値なし、10 クラス以下の 18 データセットで、XGBoost などのブースティング木を明確に上回った。
 - ・ 複雑な AutoML と同等だった。速度は CPU で約 230 倍、GPU で約 5,700 倍。
 - ・ 別の 67 データセットでも、同じ結果を確認した。

時間 (Tune + Train + Predict) については以下参照 (論文記載)

手法	時間(秒)
LightGBM	3280
CatBoost	3746
XGBoost	3364
Auto-sklearn 2.0	3601
AutoGluon	3077
TabPFN n.e.(アンサンブルなし)	1.301(CPU)/0.0519(GPU)
TabPFN(32通りの順列でアンサンブル)	37.59(CPU)/0.6172(GPU)
TabPFN + AutoGluon	3109(CPU)/3077(GPU)

Tune + Train + Predict において、Tune は以下で行った。(論文記載)

グループ	Tune の方法	書かれている場所
LogReg、 KNN、 XGBoost、 LightGBM、 CatBoost	ランダムサーチ。ランダムに引いた設定を5分割交差検証で評価し、時間予算を使い切るか10,000設定に達するまで続ける。ROC AUC が最良の設定を選び、訓練データ全体で再学習する	Sec.5.2 の Baselines
AutoGluon、 Auto-sklearn 2.0	各システムの内部の仕組み。本文には、Auto-sklearn 2.0 はベイズ最適化で、評価したモデルを重み付きアンサンブルにする、AutoGluon は木やニューラルネットワークなどを積み重ねたアンサンブルにする、という程度の説明しかない	Sec.5.2 の Baselines

対象は、TabPFN n.e.(アンサンブルなしの版) です。1つのデータセットの予測に、CPU で平均1.30 秒、GPU で0.05 秒かかります。

この時間は、5分でチューニングした最も強い比較手法と、同等の性能です。

(例)

以下の表ののような表形式のデータセットがあったとする。

	年齢	年収(万円)	信用度
1行目	25	300	低
2行目	45	800	高
3行目	35	500	低
4行目	50	900	高
5行目	30	350	?
6行目	48	850	?

Tabfm は、上記の表形式のデータセットから、1～4 行目までをみて、5 と 6 行目の信用度について推論を行うことができる。ポイントは、その際に学習を行わない点。

INTRODUCTION

画像や文章では深層学習が主役ですが、表では XGBoost などの木の手法が今も強い、という現状の確認です。

表形式データの分類のやり方に、根本的な変化を提案する。新しいデータセットの訓練部分に、モデルをゼロから当てはめる (fit) ことはしない。代わりにこの工程を、表形式データセットの事前分布から人工的に作った分類タスクを解くように事前学習した大きな Transformer による、1 回の順伝播に置き換える。

本手法は事前分布データ適合ネットワーク (PFN、Müller et al., 2022、Sec.2 参照) に基づく。PFN は、訓練と予測のアルゴリズムそのものを学習する。PFN は、サンプリングできる任意の事前分布のもとでベイズ推論を近似し、事後予測分布 (PPD) を直接近似する。NN や GBDT の帰納バイアスは、実装が効率的であること (L_2 正則化、ドロップアウト、木の深さ制限など) に依存する。PFN では、望ましい事前分布を組み込んだデータ生成アルゴリズムを設計すればよい。これは、学習アルゴリズムの設計の仕方を根本的に変える。

練習用の表は、BNN(ランダムなニューラルネットワークから作る表) と SCM(原因から結果へのつながりから作る表) で生成します。

単純な構造ほど選ばれやすくします (オッカムの剃刀)。

表の作り方の細かい数値 (ノード数など) も、固定せず確率分布から引きます。そのため、いろいろな表が練習に出てきます。

「無限のアンサンブル」は、「ありうるデータの作られ方すべてを、確からしさを重み付けして平均したような答え」という意味です。それを 1 回の計算で出せるように学習します。交差検証やモデル選択が要らないのは、この平均が、最初から答えに織り込まれているからです。

TabPFN が間違えるデータと、木の手法が間違えるデータが別々だ、という意味です。両

方を組み合わせると、お互いの弱点を補えます

ポイント

『PFN は、SCM の為、サンプリングできる任意の事前分布のもとでベイズ推論を近似し、事後予測分布 (PPD) を直接近似する』

BACKGROUND ON PRIOR-DATA FITTED NETWORKS (PFNS)

仮説 ϕ は「表を作る1つの仕組み」です。SCM を使う場合は、1つの SCM(設計図) が1つの仮説です。 Φ は、ありうる SCM 全体です。前回の整理どおり、SCM は設計図で、表そのものではありません。次のステップでは、この仮説を使って予測します。

データセットには、正解ありの行 (D_{train}) と、正解が隠れた行があります。これまでの1〜4行目と5〜6行目です。

PPD は、訓練データを見たうえでの、テストの行の答えの確率です。記号 $p(\cdot|x_{\text{test}}, D_{\text{train}})$ の縦線 $|$ は「 $\sim$ が与えられたもとで」の意味です。

式 (1) は、全仮説を重み付きで足し合わせています。各仮説について3つを掛けます。

- $p(y|x, \phi)$: その仮説のもとで、答えが y になる確率
- $p(D|\phi)$: その仮説のもとで、手元の訓練データが出る確率
- $p(\phi)$: その仮説の事前の確からしさ

$\int$ は全仮説にわたる合計で、 α は比例です。最後に、答えの確率の合計が1になるよう調整します。式の D は、 D_{train} のことです。

公式の理解

(1) の公式: $p(y|x, D)$: 事後予測分布 (PPD) の式 (理想の式)

$$p(y|x, D) \propto \int_{\Phi} p(y|x, \phi) p(D|\phi) p(\phi) d\phi.$$

1. 訓練データ(例の行)は4行です。

x1	y
0.5	1
-1.2	0
0.3	1
-0.4	0

予測したい行は1行で、 $x_1=0.4$ のときの y が知りたい、という状況だとする。

仮説は3つで、最初の確からしさ ($p(\phi)$) は、どれも $1/3$ です。

- H_1 : x_1 が正なら $y=1$ 、負なら $y=0$
- H_2 : x_1 が負なら $y=1$ 、正なら $y=0$
- H_3 : y は x_1 と無関係で、ランダム (1か0が半々)

表の各列の意味

列	意味	H_1	H_2	H_3
$p(\phi)$	データを見る前の確からしさ	$1/3$	$1/3$	$1/3$
$p(D \phi)$	その仮説のもとで、上の4行の y が出る確率	1(4行とも合う)	0(4行とも合わない)	$1/16$
$p(y=1 x,\phi)$	その仮説のもとで、 $x_1=0.4$ の y が1になる確率	1	0	$1/2$
3つの積	上の3つを掛けたもの	$1/3 = 32/96$	0	$1/96$

論文内 (1) の公式に当てはめると

式(1)

$$p(y | x, D) \propto \int p(y | x, \phi) \cdot p(D | \phi) \cdot p(\phi) d\phi$$

仮説が H1、H2、H3 の3つだけなので、積分($\int$)は3つの足し算(Σ)になります。

$$p(y | x, D) \propto \Sigma(H1 \sim H3) p(y | x, \phi) \cdot p(D | \phi) \cdot p(\phi)$$

この例での記号は次のとおりです。

- x: 予測したい行の $x_1=0.4$
- D: 1～4行目の例
- y: 知りたい答え(1 か 0)

(2) の公式の理解

$$\mathcal{L}_{PFN} = \mathbb{E}_{(\{(x_{test}, y_{test})\} \cup D_{train}) \sim p(D)} [-\log q_{\theta}(y_{test} | x_{test}, D_{train})]. \quad (2)$$

基本的な考えは、ロジスティック回帰 (二値) のクロエン

ただ、ロジとかと比較した際に違う点がある。

違い2: 損失を、何にわたって平均するかが違います。

- ロジスティック回帰: 1つの表の、訓練データの行にわたる平均
- PFN: 式 (2) の E は、無数の合成表と、その表の隠し方にわたる平均

	損失	学習の単位
ロジスティック回帰	交差エントロピー	1つの表 → その表専用の重み
PFN(式2)	交差エントロピー	数億の表 → どの表にも使える手順

仮説	事前確率	尤度	掛け算(÷3を省略)	重み(合計で割る)
H_2	1/3	0.05	0.05	$0.05 \div 0.10 = \mathbf{0.5}$
H_3	1/3	0.03	0.03	$0.03 \div 0.10 = \mathbf{0.3}$
H_4	1/3	0.02	0.02	$0.02 \div 0.10 = \mathbf{0.2}$

事前確率がすべて同じなので、重みは、尤度の比になります。この「事前×尤度」が、式(1)の $p(D|\varphi)p(\varphi)$ の部分です。
→次:この重みで、予測を平均します。

(1) と (2) の自分の理解。

武器

1 : $p(\varphi)$:仮説 (SCM) を1つ選ぶ確率のルール。

2 : $p(D|\varphi)$:選んだ SCM から表を作る仕組み。

3 : $p(y|x, \varphi)$: 仮説のもとで x (: 答えがマスクされた部分の特徴量) が与えられたもとで、 y が出る確率

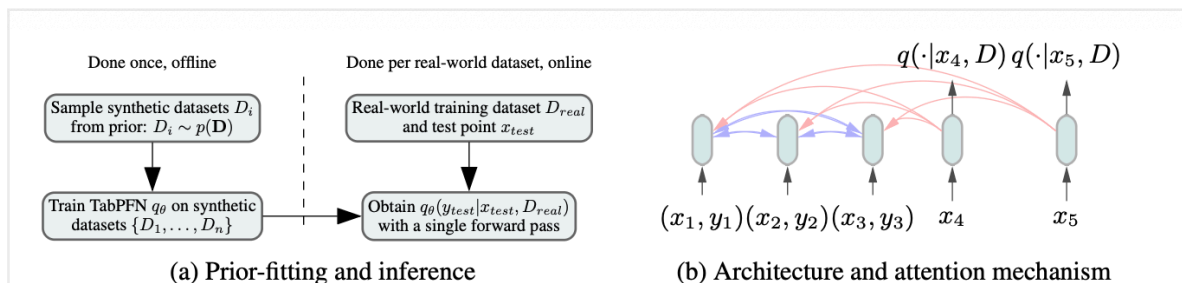
この1、2が事前確率となる。事前確率がわかると、(1) の公式は、組み立てることができる。

ただ、SCM 自体のルールの決め方は無限にあり、各 SCM をどのくらい選らずのか、またノイズの発生方ほは無限に存在するため、計算ができない。だから、(2) の公式を使って近似させる。

(2) の式では、実際に作った表から数値を読み解き、誤差を修正するを何回も繰り返し近似させる。

更新されるのは θ (transformer の重み) の部分。($q\theta$ は確率を表す。例 : 確率 60% なら、 $q\theta = 0.6$)

ロジの確率に変化する前の部分の式を思い出す。



図の解釈

(a) 先のどの公式 (1)、(2) の内容を図示している・左側がscm を元に行を生成し表の作成。その表を元に、 PFN で学習を進めていく。次に右に移り、答えのマスクされたもので推論を行う。

(b) 推論の詳細。各行ごとを1トークンとし、アテンションで、トークンごとの相関を含めたベクトルに変換。ただ、答えがマスクされたトークン同士のアテンションは計算しない。それにより、計算を軽くしている。

3 THE TABPFN: A PFN FITTED ON A NEW PRIOR FOR TABULAR DATA

(本文)

私たちのTabPFNは、事前分布データ適合ネットワーク (Prior-data Fitted Network, PFN; Sec.2 参照) であり、Sec.4 で導入する表形式データ向けの新しい事前分布からサンプリングしたデータで適合 (fit) される。元のPFNアーキテクチャ (Müller et al., 2022) には、2点の変更を加えた。i) アテンションマスク (attention masks) にわずかな変更を加え、推論時間を短縮した。ii) さらに、ゼロパディング (zero-padding) によって、特徴量の数が異なるデータセットでもモデルが動くようにした。これらのアーキテクチャ上の改良は、Müller et al. (2022) と比べた主な貢献とともに、付録E.2で詳述する。

事前分布への適合 (prior-fitting) の段階では、Sec.4 で述べる事前分布からのサンプルで、TabPFNを一度だけ学習させる。より正確には、12層のTransformerを、合成的に生成した512個のデータセットからなるバッチ (batch) 18,000個分で学習させた。これには、8基のGPU (Nvidia RTX 2080 Ti) を積んだ1台のマシンで、合計20時間を要した。これにより、すべての評価に使う単一のネットワークが得られた。この学習ステップは中程度に高コストだが、オフライン (offline) で、事前に、しかもTabPFNにつき一度だけ、アルゴリズム開発の一部として行われる。本論文のすべての実験で、同じTabPFNモデルを使う。学習の詳細は付録Eにある。

推論時には、TabPFNは、私たちのデータセット事前分布に対するPPDを近似する。すなわち、SCMとBNNの空間 (Sec.4 参照) にわたる周辺予測 (marginal predictions) を近似する。これには、データに対する単純で因果的な説明への偏り (bias) も含まれる。実験では、TabPFNの1回の順伝播による予測と、32回の順伝播をアンサンブルした予測を示す。後者では、べき乗変換 (power transformation; 確率0.5で適用) と、特徴量列およびクラスラベルのインデックスの回転 (rotating the indices) を施したデータセットを使う (詳細は付録C.2.6)。

脚注1 (PDFでは5ページ目): TabPFNの学習中にすでに特徴量列とクラスラベルのインデックスを回転させているので、原理的には、これら2つの回転に対して不変な表現を学習できる。学習時間を長くするか、より大きなTransformerを使えば、TabPFNはこれらの不変性を学び、回転が不要になると期待している。

図2のキャプション: 私たちの事前分布でデータを生成するグラフの概観。入力 x は、観測されない (unobserved) ノード z を通じて出力 y に写像される。図はMüller et al. (2022) に基づく。(a) BNN、(b) SCM、(c) 事前分布からサンプリングしたSCM。

(解説)

Sec.3は、TabPFN固有の変更点、学習の設定、推論の設定をまとめた節です。事前分布の中身 (SCMやBNN) は、Sec.4で詳しく説明されます。

元のPFNアーキテクチャ (Müller et al., 2022) には、2点の変更を加えた。i) アテンショ

ンマスク (attention masks) にわずかな変更を加え、推論時間を短縮した。ii) さらに、ゼロパディング (zero-padding) によって、特徴量の数が異なるデータセットでもモデルが動くようにした

変更点2つ

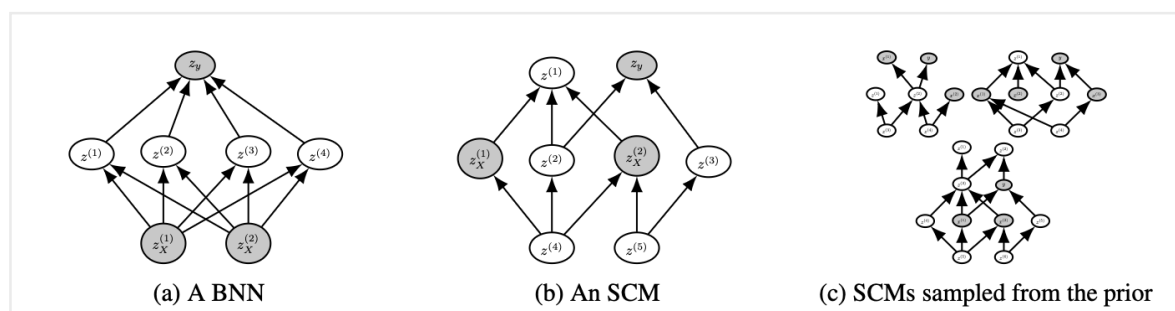
(i) マスクの変更:付録に、訓練行同士の self-attention と、テスト行から訓練行への cross-attention を、2つのモジュールに分けたとあります (E.2)。attention 行列の大きさも、 $(n+m)^2$ から $n^2+n \times m$ に縮んでいます (C.2.6 の iv)。n は訓練点、m は推論点の数です。これは、図1(b)の「テストは訓練だけを見る」に対応します。

(ii) ゼロパディング:特徴量の数 k が最大数 K より少ないときは、足りない列を0で埋めまます。さらに、元の特徴量を K/k 倍して、値の大きさをそろえます (E.2)。

実際の学習学習

- ・ 12層のTransformerを、18,000バッチ $\times$ 512個の合成データセット = 9,216,000個で学習します (付録E.3に同じ数字があります)。
- ・ 1層のTransformerとは、attention + FFN (全結合層：512次元 $\rightarrow$ 1024次元 $\rightarrow$ 512次元) + 残差接続のこと。
- ・ 1バッチは、512個のデータセット (512枚の表) のことで、512行ではありません。合成データセット1つの大きさは1,024行です (付録E.3)
- ・ 所要時間は、8GPUで20時間です。モデルのパラメータ数は25.82Mです (付録E.1)
- ・ 各ステップで事前分布から新しくサンプリングして作ります。使い回しの有無は、論文の本文には明記されていません。

数字	何を数えているか	場所
埋め込みサイズ512	1トークン(1行)を表す数字の個数	付録E.1
FFN隠れ層1,024	FFNの途中の幅	付録E.1
バッチ512	1回の更新で見る表の枚数	付録E.3
表の大きさ1,024	合成した表1枚の行数	付録E.3



図の解釈

図自体は、表の作成方法を図示したもの。

(a) では、BNN の元、すべての仮説がわかっており、それにより順番で値を算出し行を作った。

(b) では、SAM の元行の作成。

(c) では、(b) のもとでの表の作成方法

白色が、観測されないノード (変数：表でないの値)

灰色が、観測されるノード (変数：表で出る値。)

4 A PRIOR FOR TABULAR DATA

(本文)

私たちの手法の性能は、適切な事前分布 (prior) をどう指定するかに決定的に依存する。PFN は、この事前分布に対する PPD を近似するからである。Sec.4.1 では、私たちの事前分布の基本的な技法を概説する。すなわち、事前分布のほぼすべてのハイパーパラメータについて、点推定 (point-estimates) ではなく分布を用いる。Sec.4.2 では、事前分布に単純さ (simplicity) を取り入れる動機を述べる。Sec.4.3 と 4.4 では、構造的因果モデル (Structural Causal Models, SCM) とベイジアンニューラルネットワーク (Bayesian Neural Networks, BNN) を、事前分布の中で多様なデータを生成する基本的な仕組み (mechanisms) としてどう使うかを述べる。私たちの SCM と BNN の事前分布は回帰 (regression) タスクしか出力しないので、Sec.4.5 で、それらを分類タスクに変換する方法を示す。表形式データ特有の性質 (相関のある特徴量とカテゴリ特徴量、指数的にスケールしたデータ、欠損値) をよりよく反映するための、事前分布への追加の改良は、付録 C.2 で述べる。

4.1 FUNDAMENTALLY PROBABILISTIC MODELS

(本文)

モデルを当てはめる (fitting) には、通常、適切なハイパーパラメータ (hyperparameters) を見つける必要がある。たとえば NN なら、埋め込みサイズ (embedding size)、層数、活性化関数 (activation function) などである。適切なハイパーパラメータを見つめるには、多くの場合、資源を大量に使う探索が必要になる。しかし、そうした探索の結果は、ハイパーパラメータの選択についての点推定 (point estimate) にすぎない。複数のアーキテクチャやハイパーパラメータ設定にわたってアンサンブル (ensembling) すると、これらのハイパーパラメータ上の分布を大まかに近似でき、性能が向上することが示されている。ただし、そのコストは、検討する選択肢の数に比例して増える。

これに対して PFN では、事前分布のハイパーパラメータについて、完全にベイズ的 (fully Bayesian) に扱える。事前分布の中のハイパーパラメータ空間 (たとえば BNN のアーキテクチャ) に確率分布を定めることで、TabPFN が近似する PPD は、この空間と、それぞれのモデルの重みにわたって同時に積分する。この考え方を、ハイパーパラメータだけでなく、異なる事前分布そのものの混合にも広げる。すなわち、BNN 事前分布と SCM 事前分布を混ぜ、その各々がさらにアーキテクチャとハイパーパラメータの混合を含む。

(解説)

従来のハイパラの探索では、モデルを作る前に、層数などのハイパーパラメータを決める必要があります。決めるために、資源を大量に使う探索を行い、最後に1つの値を選びます。これが「点推定」です。

- ・ 弱点:1つに決めると、「別の値だったかもしれない」という不確かさが捨てられます。
- ・ 対策:複数の設定で学習して、平均する (アンサンブル) と、不確かさを大まかに表せます。
- ・ 限界:選択肢が100通りなら、コストもおおよそ100倍になります (説明用の数字です)。

PNFの場合：

- ・ ハイパーパラメータそのものに確率分布を置きます。すると、式 (1) の仮説空間 Φ が、「ハイパーパラメータ×重み」まで広がります。
- ・ PPD は、その全体にわたる重み付き平均になります (あなたが整理した、仮説にわたる重み付き平均のとおりです)。

例：

新しい点xについて、「y=1になる確率」を、各仮説が出します。これは、式(1)の $p(y|x, \phi)$ です。

仮説	P(y=1)
H ₂	0.6
H ₃	0.7
H ₄	0.9

4. 重みの作り方(今回は省略していた部分)

前回の例では、「データを見たあとの重み 0.5、0.3、0.2」を、いきなり出していました。実際は、次の3段階で作ります。

1. 事前確率 $p(\varphi)$: データを見る前の信じやすさです。ここでは、3つとも同じ $1/3$ とします。
2. 尤度 $p(D|\varphi)$: 手元のデータ D が、その仮説のもとで、どれだけ出やすいかです。ここでは、0.05、0.03、0.02 とします。
3. 掛けて、合計が1になるよう直す:

仮説	事前確率	尤度	掛け算($\div 3$ を省略)	重み(合計で割る)
H_2	$1/3$	0.05	0.05	$0.05 \div 0.10 = \mathbf{0.5}$
H_3	$1/3$	0.03	0.03	$0.03 \div 0.10 = \mathbf{0.3}$
H_4	$1/3$	0.02	0.02	$0.02 \div 0.10 = \mathbf{0.2}$

事前確率がすべて同じなので、重みは、尤度の比になります。この「事前 $\times$ 尤度」が、式(1)の $p(D|\varphi)p(\varphi)$ の部分です。

→ 次: この重みで、予測を平均します。

重み付き平均

各仮説の予測に、重みを掛けて足します。

$$0.6 \times 0.5 + 0.7 \times 0.3 + 0.9 \times 0.2 = 0.30 + 0.21 + 0.18 = \mathbf{0.69}$$

これが、全仮説を混ぜた予測 (PPD) の $P(y=1)$ です。式 (1) の積分の、離散版 (足し算) にあたります。

確認と

4.2 SIMPLICITY

私たちは、事前分布を、オッカムの剃刀 (Occam's Razor) や速度事前分布 (Speed Prior) に述べられているような、単純さの考え方に基づいて作る。競合する仮説を考えるときは、より単純なものが好まれる。認知科学の研究でも、人間の思考にこうした単純な説明への選好があることが明らかにされている。ただし、単純さの概念はどれも、単純さを何で定義するかという特定の基準の選択に依存する。以下では、SCMとBNNに基づく事前分布を導入し、そこで単純さを、ノード数やパラメータ数の少ないグラフとして実装する。

4.3 SCM PRIOR

(本文)

因果的な知識 (causal knowledge) は、半教師あり学習 (semi-supervised learning)、転移学習 (transfer learning)、分布外汎化 (out-of-distribution generalization) を含

む、さまざまなMLタスクを促進しうることを示されている。表形式データには、しばしば列どうしの因果関係が見られ、因果的な仕組み (causal mechanisms) は、人間の推論における強い事前分布であることが示されている。そこで、私たちは、TabPFNの事前分布を、因果関係をモデル化するSCMに基づけた。

SCMは、構造的代入 (structural assignments;メカニズム (mechanisms) と呼ぶ) の集合 $Z := (\{z_1, ..., z_k\})$ から成り、 $z_i = f_i(z_{PA_G(i)}, \epsilon_i)$ と表される。ここで、 $PA_G(i)$ は、基礎にあるDAG G (因果グラフ, causal graph) における、ノード i の親 (その直接の原因) の集合である。 f_i は (非線形でありうる) 決定的な (deterministic) 関数で、 ϵ_i はノイズ変数である。 G における因果関係は、原因から結果へ向かう有向辺 (directed edges) で表され、各メカニズム z_i は G のノードに割り当てられる。図2に、これを図示してある。

(解説：ここでの主張は以下の表にまとめられる。)

記号	意味
z_i	ノード i の値 (1つの変数)
f_i	z_i を決める関数。「決定的」=同じ入力なら同じ出力
$PA_G(i)$	ノード i の親 (直接の原因) の集合
ϵ_i	ノイズ (乱数)。ランダムさは、すべてここに入る
G	DAG (因果グラフ)。矢印は原因→結果

DAG は、有向非巡回グラフ (directed acyclic graph) の略です。

Predictions based on ideas from causal reasoning

因果的な推論の考え方にに基づく予測

(本文)

先行研究では、因果推論 (causal inference) を用いて、未見のデータ上の観測を予測するために、因果的な推論 (causal reasoning) を応用してきた。因果推論とは、介入 (interventions) と観測データ (observational data) を用いて、システムの構成要素間の因果関係を特定しようとする方法である。予測された因果的な表現 (causal representations) は、その後、新しいサンプルに対する観測的な予測を行うため、または説明可能性 (explainability) を与えるために使われる。既存の研究の多くは、下流 (downstream) の予測に使う単一の因果グラフを決めることに集中している。これは問題になりうる。ほとんどの種類のSCMは、介入データなしには識別不能 (non-identifiable) であり、DAGの空間の組合せ的な性質のために、整合するDAGの数が爆発するからである。最近、Ke et al. (2022) と Lorch et al. (2022) は、観測データと介入データから因果グラフを近似するために、Transformerを使った。私たちは、推論のステップで、明示的なグラフ表現をすべて省き、PPDを直接近似する。したがって、因果推論は行わず、下流の予測タスクを直接解く。SCMのような過程が私たちのデータを生成しているという、この暗黙の仮定は、Pearlの「因果のはしご (ladder of causation)」で説明

できる。これは推論の種類の抽象化で、上の段 (rung) ほど、より込み入った推論の概念を表す。最下段には関連 (association) があり、これは ML の大部分を含む。2 段目は、介入の効果の予測、すなわち、特徴量に直接働きかけたときに何が起きるかを考える。私たちの研究は、Kyono et al. (2020; 2021) と同様に「1.5 段目」とみなせる。因果的な推論は行わず、SCM が一般的なデータセットをうまくモデル化していると仮定して、観測データ上で、関連に基づく予測を行う。付録 B の図 8 では、私たちの (原文は、ここで名詞が抜けています) が、単純な SCM 仮説と確かに整合することを、実験で示す。

(解説)

簡単に言うと、これまで解説してくれた、表の作成においては scm を使って因果関係から作成しているが、学習時には、この因果関係を渡さず、観測された特徴量と正解ラベルのみを使って学習する。

補足：因果関係は渡さないが、影響は残る

因果関係のグラフそのものは渡しません、大量の表を通して、SCM の癖が、 θ の中に染み込みます (私の読みです)。論文にも、それを示す記述があります。

- Sec.3 に、TabPFN の予測は、単純で因果的な説明への偏りを含む、とあります。
- 付録 B.3.1 に、TabPFN の予測が滑らかで、単純な SCM を好む事前分布によるものだろう、とあります。

つまり、「因果関係を明示的に教える」のではなく、「因果的な仕組みから作った表を大量に見せる」ことで、その性質を暗黙に学ばせています。

Defining a prior based on causal models

因果モデルに基づく事前分布の定義

(本文)

SCM に基づく PFN 事前分布を作るには、教師あり学習タスク (すなわちデータセット) を作るサンプリング手順 (sampling procedure) を定義する必要がある。ここでは、各データセットは、ランダムにサンプリングした 1 つの SCM (DAG 構造と決定的関数 f_i を含む) に基づく。SCM が与えられたら、因果グラフ G のノードの集合 z_X を、合成データセットの各特徴量に 1 つずつサンプリングし、さらに G からノード z_y を 1 つサンプリングする。これらのノードは観測ノード (observed nodes) である。 z_X の値は特徴量の集合に含まれ、 z_y の値は目的変数 (targets) として働く。そのような SCM とノードのリスト z_X 、 z_y のそれぞれについて、 n 個のサンプルは、SCM 中のすべてのノイズ変数を n 回サンプリングし、それらをグラフを通して伝播させ (propagating)、 n 個すべてのサンプルについて z_X と z_y の値を取り出すことで生成される。図 2b は、観測された特徴量ノードと目的ノードを灰色で示した SCM を描いている。得られる特徴量と目的変数は、生成する DAG 構造を通じて相関する。これにより、特徴量は、順方向と逆方向の因果 (forward and backward causation) を通じて条件付き従属 (conditionally dependent) になる。すなわち、目的変数は、特徴量の原因にも結果にもなりうる。図 3 では、2 つの異なる SCM から生成したサンプルを実際のデータセットと比べ、私たちの事前分布がモデル化できるデータセットの空間の多様性を示している。

(解説)

PFNの事前分布とは、「データセットを作る手順」です。

1つの表につき、1つのSCM

各データセット(表)は、ランダムに引いた1つのSCMから作ります。DAGの形と、各 f_i も、そのとき一緒に決まります。表を1枚作るたびに、SCMを引き直します。

注意 (読み違いやすい点):行ごとにSCMが変わるのではありません。1つの表の中では、SCMは同じで、行ごとに変わるのはノイズだけです

n行を作る

手順は、次のとおりです。

1. ノイズ変数を、すべてのノードについて引く(1行ぶん)。
2. 根(親のないノード)から順に、式を計算して、値をグラフに流す(伝播させる)。
3. 観測ノードの値だけを取り出す。これが1行になる。
4. 1~3を、n回くり返す(ノイズを毎回引き直す)。

「順方向・逆方向の因果」

- 表の x_1 , x_2 , y は、矢印(と隠れノード a)を通して、互いに関わります。論文は、これを「DAG構造を通じて相関する」と述べています。
- 「順方向・逆方向の因果」は、続く i.e. の文から、「ラベルが、特徴量の原因にも結果にもなりうる」という意味に読めます(私の読みです)。

上の表で確認します。選び方を変えると、同じSCMから、向きの違う問題が作れます。

- **選び方 A:**特徴量が (x_1, x_2) で、ラベルが y です。ラベルは、特徴量の結果です($x_2 \rightarrow y$)。
- **選び方 B:**特徴量が (x_2, y) で、ラベルが x_1 です。ラベルは、特徴量の原因です($x_1 \rightarrow x_2$)。

4.4 BNN PRIOR

(本文)

私たちは、Müller et al. (2022) が導入したBNN事前分布も考え、これを、上述のSCM事前分布と混ぜる。混ぜ方は、PFNの学習中に、データセットを、どちらか一方の事前分布から、等しい確率でランダムにサンプリングする、というものである。BNN事前分布からデータセットをサンプリングするには、まず、NNのアーキテクチャ(architecture)とその重み(weights)をサンプリングする。次に、これから生成するデータセットの各データ点について、入力 x をサンプリングし、それをサンプリングしたノイズ変数とともにBNNに通し、出力 y を目的変数として使う(図2a参照)。このBNN事前分布から、SCMベースとBNNベースの両方を混ぜた最終的な事前分布までを比べる、付録B.4の実験的な

アブレーション (ablation) は、私たちの新しい SCM ベース事前分布の強さを示している。

(解説)

BNN 事前分布と SCM 事前分布を混ぜる」と予告されていました。4.4 は、その混ぜ方の具体的な説明です。

- **混ぜる単位は、データセット (表) 1 枚ごと**です。表を作るたびに、SCM か BNN かを、等確率 (1/2 ずつ) で選びます。1 枚の表の中で、行ごとに混ぜるわけではありません。
- 付録の Table 5 にも、「Sample SCM vs BNN」を、一様に選ぶ項目 (Uniform Choice) があります。

(それぞれの作り方は、図 2 参照)

BNN から表を作る

手順は、次のとおりです。

1. NN のアーキテクチャと重みをサンプリングする (表 1 枚につき 1 回)。
2. 各行について、入力 x をサンプリングする。
3. x を、ノイズ変数とともに BNN に通し、出力 y をラベルとして使う。
4. 2～3 を、 n 回繰り返す。

BNN と SCM の違い (図 2 の a と b を比べます)

	BNN (図 2a)	SCM (図 2b)
入力 x の作り方	x を直接サンプリングして、NN に通す	ノイズを引き、根から順に流す。グラフのノードから、特徴量を選ぶ
特徴量とラベルの向き	特徴量が原因で、ラベルが結果 (固定)	ラベルは、特徴量の原因にも結果にもなる
隠れ部分	層ごとの隠れユニット	グラフ内の隠れノード

(ablation)

最後に、BNN だけで表を作って学習・推論した場合の精度の良さと、scm だけで表を作って学習・推論した場合の精度の良さと、BNN と scm で表を作って学習・推論した場合の精度の良さを検証する

事前分布	Mean CE(小さいほど良い)	Mean ROC AUC(大きいほど良い)
BNNのみ	0.811±0.009	0.865±0.007
SCMのみ	0.771±0.006	0.881±0.002
SCM+BNN	0.776±0.009	0.883±0.003

評価指標

Mean CE：クロエンの誤差（小さい方がいい。）

Mean ROC SUC :推論とか関係なしに確率的に、答えのラベルを均等に算出した時（確率 0.5）と比較してのスコア（1 が最も良い、0.5 がランダムに行った時、0 が最も悪い）

図の解釈

(図3)：合成データと実データの見比べ

(a) scm で作った表と、(b) 実データで作った表がどれくらい似ているかを可視化したもの（判断基準は願力）

ともに、左が2値分類、右がサンクラス分類。特徴量4つの、全ペアのため、それぞれに6個の散布図がある。

言えること:事前分布は、実データのような多様な散布図を作れる、という論文の主張の、見た目の裏づけです。「どちらが合成か、見分けにくい」ことがねらいだと思います

図4:決定境界の比較

データに対する分類の境界線の可視化。今回のデータは、iris と wine は scikit-learn 付属のデータです。

どれも、2クラス・2特徴量

4.5 MULTI-CLASS PREDICTION

(本文)

多クラス予測

ここまでに述べた事前分布は、スカラー(連続値)のラベルを返す。不均衡な多クラスデータセットに対する、合成の分類ラベルを作るには、スカラーのラベル $\hat{y}$ を、離散的なクラスラベル y に変換する必要がある。これは、 $\hat{y}$ の値を、クラスラベルに対応する区間(intervals)に分割することで行う。

- i) クラス数 N_c を、 $N_c \sim p(N_c)$ からサンプリングする。ここで $p(N_c)$ は、整数上の分布である。
- ii) $N_c - 1$ 個のクラス境界(class bounds) B_i を、連続値の目的変数 $\hat{y}$ の集合から、ランダムにサンプリングする。
- iii) 各スカラーラベル $\hat{y}_i$ を、それを含む唯一の区間のインデックスに写像する。 $y_i \leftarrow \sum_j [B_j < \hat{y}_i]$ 。ここで $[\cdot]$ は指示関数(indicator function)である。

例えば、 $N_c = 3$ クラスで、境界が $B_c = \{-0.1, 0.5\}$ なら、3つの区間 $\{(-\infty, -0.1], (-0.1, 0.5], (0.5, \infty)\}$ が定まる。任意の $\hat{y}_i$ は、 -0.1 より小さければラベル0に、 $(-0.1,$

0.5] に入ればラベル1に、それ以外はラベル2に写像される。最後に、クラスのラベルをシャッフルする。すなわち、範囲に関するクラスラベルの順序を取り除く。

(図5)：18個は**データセット (表)**での横軸は、学習・調整に与えた時間(1秒から1時間)と、それぞれのROC AUCのスコア。
使ったデータセットは、OpenML-CC18のデータセット。

5 EXPERIMENTS

5.1 EVALUATION ON TOY PROBLEMS

まず、私たちのTabPFNを、標準的な分類器(ハイパーパラメータ調整なし)と、図4で定性的に(qualitatively)比べる。最上段は、ノイズ付きのmoons(月形)データセットを示す。TabPFNは、サンプル間の決定境界(decision boundary)を正確にモデル化する。また、ガウス過程(Gaussian processes)と同様に、観測されたサンプルから遠い点では、不確実性(uncertainties)が大きい。2段目は、ノイズ付きのcircles(円)データセットを示す。TabPFNは、サンプルが混ざっている領域の外側のどこでも、高い確信度で、円の形を正確にモデル化する。3段目は、irisデータセットの2クラスと2特徴量を示し、4段目は、wineデータセットの2クラスと2特徴量を示す(どちらもscikit-learnにある)。これら2つの場合とも、TabPFNは、直感的で、よく校正された(well-calibrated)予測をする。

5.2 表形式MLタスクでの評価

ここから、実世界の分類タスクに対する、私たちの手法の経験的な(empirical)分析に移る。私たちの手法を、表形式データの分類における、最先端のMLおよびAutoMLの手法と比較する。

Datasets

データセット

テスト用のデータセットとして、厳選された(curated)オープンソースのOpenML-CC18ベンチマークスイート(benchmark suite)から、最大2,000サンプル(訓練用の分割では1,000)、100特徴量、10クラスを含む、すべてのデータセットを使った。得られた集合は、30個のデータセットから成る。これらを、数値特徴量のみで欠損値のない18個のデータセットと、カテゴリ特徴量、欠損値、またはその両方を含む、その他12個のデータセットに分けた。本文の図5と表1では、数値のみで欠損値のないデータセットの場合に、分析を限定する。私たちのTabPFN事前分布の開発は、この場合に焦点を当てたからである。30個すべてのデータセットの結果は、付録B.1にあり、TabPFNは、依然として強い総合的な性能を示す。ただし、カテゴリ特徴量や欠損値を持つデータセットでは、TabPFNの性能が一般に劣るため、純粋に数値の場合ほど強くはない。私たちは、小規模なデータセットに焦点を当てる。理由は、(1)小規模なデータセットは、実世界の応でよく見られる、(2)既存の深層学習(DL)手法は、この領域で最も制限される、(3)TabPFNは、より大きなデータセットでは、学習と評価が大幅に高コストになる(付録Aに詳述する制限)からである。

付録B.1はP16のグラフ
(グラフの見方)

3つのパネル:データセットを、3つの観点で、それぞれ2群に分けています。

- 左 (Has Categorical Features):カテゴリ特徴量を持つか
- 中 (Has Nans in Features):欠損値 (NaN) を持つか
- 右 (Multiclass):多クラス (クラスが3つ以上) か。それ以外は二値です
- 棒の長さ=Normalized ROC AUC

ただ、同じデータセット群を、3通りに切り分けたものです。ある分け方をするとき、他の性質は、群の中に混ざります。

Baselines

比較対象

私たちは、表形式データ向けの、5つの標準的なML手法と、2つの最先端のAutoMLシステムと比較する。MLモデルとして、単純で高速な2つのベースライン、K近傍法 (K-nearest-neighbors, KNN) とロジスティック回帰 (Logistic Regression, LogReg) を検討した。さらに、3つの人気のある木ベースのブースティング (boosting) 手法、XGBoost、LightGBM、CatBoostを検討した。各MLモデルについて、5分割交差検証 (5-fold cross-validation) を使って、ランダムに引いたハイパーパラメータ設定を評価した。これは、与えられた予算を使い切るか、10,000個の設定を評価し終えるまで続けた (探索空間は付録F.2を参照)。その後、最も成績のよいハイパーパラメータ設定 (ROC AUC OVOが最大のもの) を選び、訓練データ全体で再学習 (refit) した。必要な場合は、欠損値を平均で補完し (imputed)、カテゴリ入力をone-hotまたは順序 (ordinal) エンコードし、特徴量を正規化し、カテゴリ特徴量の指標 (indicators) を渡した。より複雑だが強力なベースラインとして、2つの最先端のAutoMLシステムを選んだ。AutoGluonは、NNや木ベースのモデルを含むMLモデルを、スタック型アンサンブル (stacked ensemble) にまとめる。Auto-sklearn 2.0は、ベイズ最適化 (Bayesian Optimization) を使い、評価したモデルを、重み付きアンサンブルにまとめる。²

過去の研究では、小～中規模の表形式データ (1万サンプル未満) では、DLのベースラインは、GBDTやAutoMLの手法を上回らず、同等にもならないことが分かっている (より大きなデータでは、GBDTと同等)。さらに、TabNet、SAINT、Regularization Cocktails、Non-parametric Transformersのような、DL手法は、はるかに大きなデータセットで評価されており、独自のパラメータ調整や前処理を使うことが多い。それでも、代表的な2つのDL手法、Regularization Cocktails³とSAINTを評価する。結果は次。

列	中身	出典
TabPFN n.e.	アンサンブルなしの、速い版	表1のキャプション
TabPFN	32回の順伝播をアンサンブルした、標準の設定	Sec.3、Sec.5.2のEvaluation Protocol
TabPFN + AutoGluon	TabPFNとAutoGluonの予測を平均したもの	Sec.5.2のEnsembling

AutoGluonとは：

Amazon(AWS)が中心となって開発した、オープンソースのライブラリです。データと時間の予算を渡すだけで、AutoGluonが、内部で複数のモデルを、学習・調整・組み合わせして、予測を出します。

TabPFN + AutoGluonに関して、平均しているのは、**精度や時間ではなく、各行に対する予測(確率)**です。

Evaluation Protocol

評価プロトコル

(本文)

各データセットと各手法について、5回の反復(repetitions)を評価した。各反復は、異なる乱数シード(random seed)と、訓練・テスト分割を持つ(訓練50%、テスト50%のサンプル。同じシードなら、すべての手法が同じ分割を使った)。データセット全体にわたって結果を集約するために、ROC AUC(多クラス分類ではone-vs-one(OVO))の平均、順位(ranks)、勝ち数(wins)を、95%信頼区間(confidence interval)とともに報告する。そして、{30, 60, 300, 900, 3600}秒の予算(budget)での、ベースラインの性能と比較する。⁴ 私たちのTabPFNは、Sec.3で述べたとおり、32個のデータの並べ替え(permutations)をアンサンプルに使う。並べ替えなしのTabPFNも評価し、表1では「TabPFN n.e.」と表記する。

(ポイント)

:5回の反復と、同じ分割

- **乱数シード**:乱数の出方を決める番号です。同じシードなら、同じ結果が再現できます(一般的な説明です)。
- **5回の反復**:シードを5通りにして、そのたびに、行をランダムに、訓練50%とテスト50%に分けます。
- **同じ分割**:あるシードでは、すべての手法が、同じ分割で評価されます。

説明用に作った例です。表が800行のとき、シード0では、ランダムに選んだ400行が訓練、残り400行がテストです。シード1では、別の400行が、訓練になります。手法A、Bは、どちらも、シード0の分割で比べられます。

同じ分割で比べるので、「たまたま簡単な分割に当たった」という運の差が、手法間には出ません。

注意:前に

Baselines

(本文)

ここで、数値のみで欠損値のない18個のデータセットに対する結果を示す。詳細は、図5と表1にある。図5は、TabPFNが、他のすべての手法よりも、精度と学習速度のトレードオフ(tradeoff)を、劇的に改善していることを示す。TabPFNは、1台のGPUで1秒未満で予測を出し、その予測は、1時間学習した最良の競合(AutoMLシステム)の性能と並び(tie)、調整したGBDT手法の性能を圧倒する(dominate)。予想どおり、単純なベースライン(LogReg、KNN)は、小さな予算でも、すでに結果を出す、全体では最も性能が悪い。GBDT(XGBoost、CatBoost、LightGBM)は、より良い性能だが、それでも、TabPFNと最先端のAutoMLシステム(Auto-sklearn 2.0、AutoGluon)には及ばない。**TabPFNは、同等の性能を持つ手法よりも、はるかに速い。**以下では、学習と予測(およ

び、該当する場合は調整) を合わせた時間を比べる。TabPFN n.e. は、1つのデータセットの予測に、CPUで平均1.30秒、GPUで0.05秒を要し⁵、5分時点の最強のベースラインと同等の性能を出す。したがって、CPUで230倍、GPUで5,700倍の高速化(speedup)になる。各手法の開発の計算コストは、無視する。その理由は、付録F.5を参照。議論した結果は、データセットにわたる集約結果(aggregate results)であり、TabPFNを含め、どの分類手法も、個々のすべてのデータセットで最良になるわけではないことを、強調したい。実際、TabPFNが、デフォルト設定のベースラインにさえ負けるデータセットも存在する。一般に、TabPFNは、カテゴリ特徴量や欠損値があるときには、あまり強くない。付録B.1は、OpenML-CC18ベンチマークの、30個のテスト用データセットすべて(カテゴリ特徴量か欠損値、またはその両方を持つ12個を含む)の結果を示す。付録B.5は、異なる種類のデータセットに対する結果の、より詳しい概観を示し、OpenMLの追加の149個の検証用データセットでの結果も含む。これは、純粋に数値のデータセットでの、TabPFNの強い性能を裏づける。また、179個すべてのデータセット(30個のテスト用と149個の検証用)の、データセットごとの結果も示す。これらは、TabPFNが、一般に、数値特徴量のデータセットでより良いが、純粋に数値でも、TabPFNがベースラインより良くないデータセットが、いくつかあり、また、TabPFNが明確に最も良いカテゴリデータセットもある、ことを示す。

(図5の具体的な見方が出てくる。)

・**左のパネル**:TabPFNは、調整をしないので、時間予算を変えても、成績は一定です。競合は、時間予算が増えると、上がっていき、1時間で、AutoMLがTabPFNの高さに並びます。本文の "tie" が、これです。GBDTは、1時間でも、TabPFNの線より下にありません。本文の "dominate" が、これです。

・**中と右のパネル**:TabPFNの線は、予算が増えると悪くなって見えます。TabPFN自身が悪化したのではありません。競合が、予算とともに強くなり、勝ち数が減り、順位が下がるためです(私の読みです)。

・**中のパネル**:データセット(18個)を使用して、「TabPFN対他のモデル」の勝敗ではなく、「全手法の中で1位」を数えている。

・**右のパネル**:平均順位

単純に順位が平均で何位であったかであるため、スコアが低い方がいい。

LogRegとKNN:本文は、「小さな予算でも結果を出す、全体では最も性能が悪い」と述べます。凡例の色が小さく、2つの線の区別は、私の読み取りでは確度が低いです。図を拡大して確認してください。

(時間の比較)

本文の基準は、「5分時点の最強のベースラインと、同等の性能を出す、TabPFN n.e.」です。つまり、比べる相手の時間は、5分(300秒)です。

TabPFN n.e.	時間	300秒との比	論文
CPU	1.301秒	約230倍	230倍
GPU	0.0519秒	約5,780倍	5,700倍

OpenML-AutoML Benchmark

(本文)

OpenML-AutoML ベンチマーク

さらに、私たちの TabPFN を、OpenML-AutoML ベンチマークの、小規模な (訓練サンプル 1,000 以下、100 特徴量、10 クラス) データセットで評価した。これらについては、外部で検証された (externally-validated) 性能の数値が、利用できる。私たちは、OpenML-AutoML ベンチマークが提供する設定を使う。すなわち、指標 (metrics)、公式の評価スクリプト、そして、さらに幅広い AutoML ベースラインについての、事前に公開された評価 (pre-released evaluations) である。1つの CPU で、1データセットあたり平均わずか 4.4 秒 (ベースラインは 60 分) で、TabPFN は、平均クロスエントロピー、正解率、OpenML メトリック⁶ の点で、すべてのベースラインを上回った。詳細な結果は、付録 B.2 に示す。

(解釈)

OpenML の 5 つのベンチマークのデータと評価指標を使って、今回のモデルの精度を検証する。ただ、5 つのうち 4 つは、論文の他の場所で、すでに使われているデータである。

対象は、5 つのデータセット

付録 B.2 と表 10 によると、次の 5 つです。前に見た、TabPFN の適用範囲 (1,000 サンプル、100 特徴量、10 クラス) に収まるものだけです。

データセット	行数	#Feat.(表の値)	クラス数
credit-g	1,000	21	2
vehicle	846	19	4
wine	178	14	2
blood-transfusion-service-center	748	5	2
Australian	690	15	2

In-depth analysis of TabPFN predictions

(本文)

TabPFN の予測の詳細な分析

私たちは、モデルの予測を、多数の方法で評価し、私たちの手法の帰納バイアス (inductive biases) など、追加の知見を得た。付録 B.3 を参照。私たちは、TabPFN が、単純な因果的説明に偏った予測をするよう学習していることを確認した (B.3.1 に詳述)。一方、GBDT 手法は、この帰納バイアスを共有しない。また、私たちの手法の、特徴量の回転に対する不変性 (invariance) と、役に立たない特徴量に対する頑健性 (robustness) も、付録 B.3.2 と B.3.3 で評価する。図 6 では、データセットにカテゴリ特徴量も欠損値もないとき、TabPFN が、ベースラインと比べて、特に強いことが観察される。

(解釈)

ここまでは、表 1 と図 5 で、「どれだけ良いか」を見ました。この段落は、「なぜそうなるか」と「どこが弱い」を調べた、付録の予告です。

付録	調べたこと
B.3.1	単純な説明への偏り(帰納バイアス)
B.3.2	役に立たない特徴量への頑健性
B.3.3	特徴量の回転への不変性
図6(B.1)	性質ごとの強さ

具体的には、上記の項目。詳細は次以降で出てくる。

Ensembling

(本文)

私たちは、TabPFNが、私たちのベースラインとは異なるデータセットで、最も良い成績を出すことを観察した。すなわち、正規化したROC AUCスコアの、データセットごとの相関(per-dataset correlation)は、TabPFNと強いベースラインとの間のほうが、強いベースライン同士の間よりも低い。これは、付録の図11に示してある。これは、私たちの手法の、新しい帰納バイアスのために、異なる予測が導かれるからだと思われる。予測をアンサンブルすることは、考慮する手法どうしの、相関した誤りが少ないほど、より効果的である(Breiman, 2001)。これは、より多様な戦略を使うことを促し、TabPFNを、ベースライン手法とのアンサンブルの理想的な候補にする(Wu et al., 2021)。また、TabPFNは、非常に速く評価できるので、ベースラインの長い実行時間と比べると、予測は、ほぼ無料である。アンサンブルの可能性を示すために、表1に「TabPFN + AutoGluon」の項目を含める。これは、TabPFNとAutoGluonの予測を平均して作ったもので、他のすべての手法を大きく上回る。

(解釈)

この段落の主張は3つ

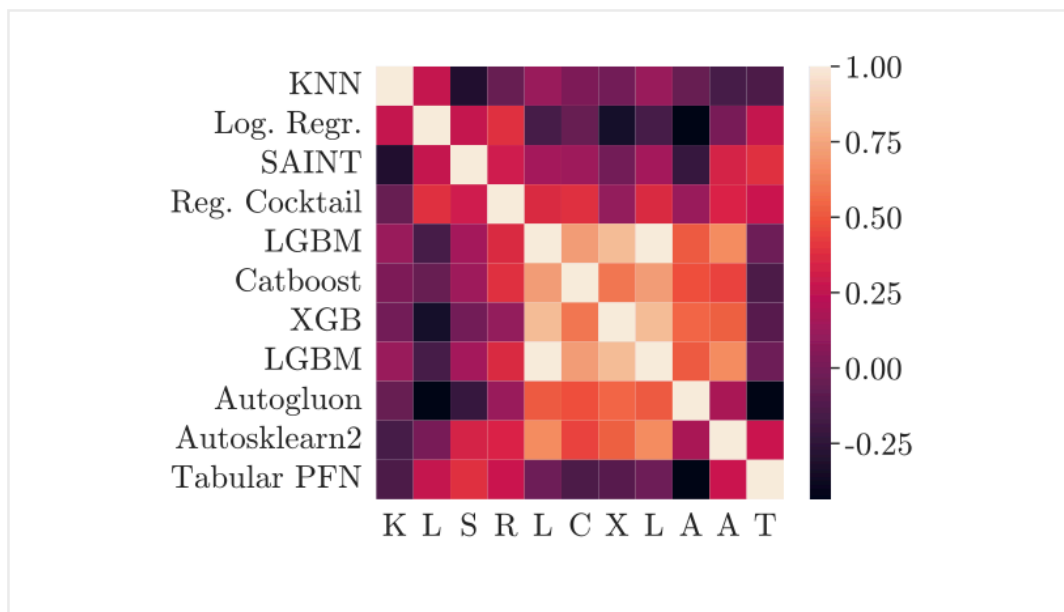
1. TabPFNは、既存の強い手法と、得意なデータセットが違う(図11)。
2. 違うので、混ぜると効果的である(誤りが違うほど、アンサンブルが効く)。
3. TabPFNは速いので、混ぜても、追加コストはほぼ無料。

そして、実例として、「TabPFN + AutoGluon」が挙げられます。

(図11の見方)

1. データセットごとに、各手法の、正規化したROC AUCを出す。

2. 手法のペアについて、「データセットを並べたとき、成績の順位が、どれだけ一致するか」を、Spearman 相関で測る。
3. 1.0 に近いほど、「同じデータセットで、良い・悪い」が一致する。0 に近いと、無関係。負なら、逆。図の色の目盛りは、-0.25 から 1.00 です。



上記のアンサンブルから、モデルによる得意不得になデータが違いため、相関関係がないもの同士をくっつけるという精度が期待できる。その際に、TabPNS は以下の資料のように、追加した時の計算時間が小さい。

	時間	AutoGluonの3,077秒に対する割合
TabPFN(GPU)	0.6172秒	約0.02%
TabPFN(CPU)	37.59秒	約1.2%
表1の「TabPFN + AutoGluon」(CPU)	3,109秒	3,109-3,077=32秒の増加
同(GPU)	3,077秒	表示の上では、増加なし

tabPNS をアンサンブルした際の結果が以下であり、わずかに性能は上昇している。

指標	TabPFN	AutoGluon	TabPFN + AutoGluon
Mean AUC OVO	0.934±.0086	0.930±.0091	0.934±.0084
Mean Acc.	0.879±.0089	0.881±.01	0.886±.0094
Mean CE(小さいほど良い)	0.716±.019	0.714±.014	0.711±.014
M. rank AUC OVO(小さいほど良い)	2.9444	4	2.6667
Mean rank Acc.	2.8889	3.8889	2.25
Mean rank CE	3.0278	3.1111	2.0833

Model Generalization

(本文)

モデルの汎化

PFNアーキテクチャは、任意の長さのデータセットを入力として受け付ける。しかし、合成の事前分布適合 (prior-fitting) の段階でモデルを学習させるとき、私たちは、合成データを最大サイズ1,024に制限した。データセットが大きいほど、事前分布適合が高コストになるからである。それでも私たちは、疑問に思った。TabPFNは、学習中に一度も見なかった、より大きな訓練セットのサイズに、汎化するだろうか。これを試すために、OpenML-AutoMLベンチマークの、18個のデータセットの集まり(表11参照)を使い、それぞれから10,000サンプルを選んだ。私たちは、最大5,000の訓練サンプルまでで、TabPFNを評価した。5回のランダムなデータ分割と、5,000のテストサンプルを使った。驚くべきことに、私たちのモデルは、学習中に見たサンプルサイズを超えて汎化する。これは、付録B.1の図10に示すとおりである。

(解釈)

これまで学習では、合成表を最大1,024行に制限しました。attentionの計算量が、行数の2乗で増えるためです(付録A)。しかし、もっと多い行数の表に対して、どの程度の精度が出てか検証した。具体的には、

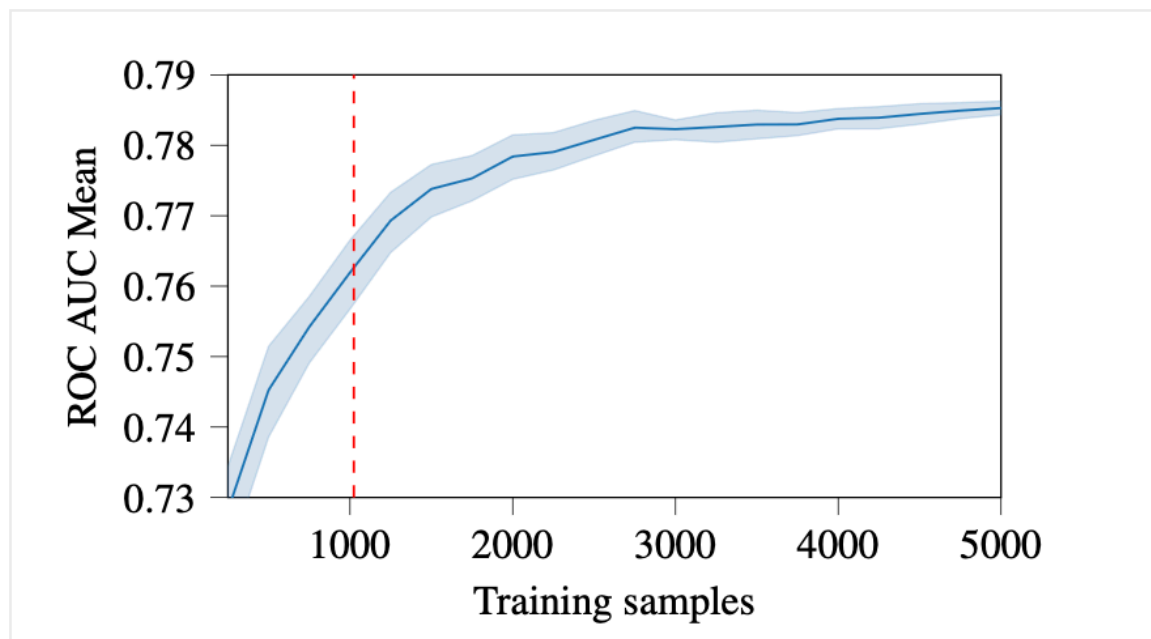
- ・ 8個の、大きなデータセットを使う。各データセットから、10,000行を選ぶ。
- ・ 5,000行をテストに固定し、訓練の行数を、少ない数から5,000まで増やす。10,000行=訓練最大5,000+テスト5,000です(AIの読みです)。
- ・ 5回、ランダムに分割し、平均する。
- ・ 指標は、図10の縦軸から、ROC AUCの平均です。

ただ、1部加工もしている。

付録F.4によると、TabPFNが扱える範囲に合わせる、次の加工をしています。

- ・ 特徴量が100より多いデータセットは、最初の100個だけを使う。
- ・ クラスが10より多いデータセットは、最初の10クラスの行だけを残す。

(結果が、P 2 1の図10)



言えること

1. 平均として、増えるほど上がる:18個のデータセットの平均で、訓練サンプルが増え
ると、ROC AUCが上がる傾向があります。
2. 学習で見たサイズを超えても、崩れない:これが、この図の主目的です。
3. 効果は逓減する:2,000あたりまでは、大きく上がります。それ以降は、緩やかで
す。3,000から5,000の増加は、約0.003と、小さな値です。

注意点：

- ・他の手法との比較がない
- この図はTabPFNだけです。GBDTやAutoMLも、訓練サンプルが増えれば、成績
は上がるのが普通です。TabPFNが、大きな表でも、競争力があるかは、この図から
は言えません。
- ・「精度」ではなく、ROC AUC
- 縦軸は、ROC AUCの平均です。正解率 (accuracy) ではありません。前に説明し
た、順位づけの良さです。

:6 CONCLUSIONS & FUTURE WORK

結論と今後の課題

(本文)

私たちは、単一のTransformerであるTabPFNが、表形式データ向けの完全なAutoML
フレームワークの仕事をするよう学習でき、0.4秒で、最良の利用可能なAutoMLフレー
ムワークが5～60分で達成する性能に匹敵する予測を出せることを示した。これは、
AutoMLの計算コストを大幅に削減し、手頃で環境に優しい (green)AutoMLを可能にす
る。

TabPFNには、依然として重要な制限がある。基礎にあるTransformerアーキテクチャ
は、付録Aで詳述するとおり、小規模なデータセットにしかスケールしない。私たちの評
価は、訓練サンプル最大1,000、欠損値のない純粋に数値の100特徴量、10クラスまでの
分類データセットに焦点を当てており、これが、(1) 大規模データセットへのスケールア

ップの研究を動機づける。TabPFNの帰納バイアスの詳細な分析(付録B.3参照)は、(2) カテゴリ特徴量のより良い扱い、(3) 欠損値、(4) 重要でない特徴量への頑健性、という拡張を指し示す。また、私たちの研究は、多数のわくわくする今後の研究を動機づける。(5) TabPFNの、既存のAutoMLフレームワークへの統合、(6) より多くの時間が与えられたときに、改善を続けるためのアンサンブル、(7) データセットに依存する事前分布の選択、(8) 表形式でないデータへの一般化、(9) 回帰タスク、さらに、信頼できるAIの次元に関するTabPFNの研究(例:(10) 分布外頑健性、(11) アルゴリズムの公平性、(12) 敵対的な例への頑健性、(13) 説明可能性)である。TabPFNの、ほぼ瞬時の最先端の予測は、(14) 新しい探索的データ分析手法、(15) 新しい特徴量エンジニアリング手法、(16) 新しい能動学習手法を、生み出す可能性も高い。最後に、因果推論における私たちの進展は、(17) SCMの分布を考慮した、介入と反事実の効果の近似についての、今後の研究を正当化する。

=====

(補足)

SCM(構造的因果モデル)

$A \rightarrow B$ 、 $A \rightarrow C$ のように、矢印でつなぐ。これがSCM(構造的因果モデル)。

①

矢印のつながり方も、各矢印の計算式も、毎回ランダムに決めます。ここでは次のように決まったとします。

- A は、ランダムな数(ノイズ)
- $B = 2 \times A + \text{少しのノイズ}$
- $C = -A + \text{少しのノイズ}$

次のステップでは、この式に値を流し込みます。

↑：これがSCM(構造的因果モデル)。↓がSCMを使った、表の作成。

②ノイズを流し込んで、1人分のデータを作成。

A にランダムな数を入れると、 B と C が式から決まります。これで A 、 B 、 C の値が1組できます。これが表の1行です。何回も繰り返せば、何行でも作れます。

③3. 表に載せる列と、当てさせる列を選ぶ。

A と C を手がかり(特徴、 x)に選び、 B を当てさせる対象(正解、 y)に選びます。 B は連続した数なので、「0より大きければ1、そうでなければ0」のように区切って、クラスに変え

ます (v1 Sec.4.5)。

x1(A)	x2(C)	y(Bから作ったクラス)
0.5	-0.5	1
-1.2	1.2	0
0.3	-0.3	1
-0.4	0.4	0

この表には「x1 が正なら y は1」という隠れた規則があります。表を作った側は、その規則を知っています。

⑤. 次の表を作るときは、つながりも計算式も作り直します

別の表では、y が x1 の2乗で決まるかもしれませんし、3列がすべて絡み合うかもしれません。1つの表ごとに、隠れた規則が違います。これを何億回も繰り返します。

それで何が身につくのか

TabPFN は、表を1つずつ見ながら「例 (正解付きの行) から、隠れた規則を読み取り、正解が隠れた行に当てはめる」練習をします。特定の規則を覚えるのではなく、規則を読み取る手順を身につけます。

実際の生成は、もっと複雑です

Nature 論文の Fig.2 と Methods には、次のような要素が書かれています。

- 数値をつなぐ計算に、小さなニューラルネットワーク、決定木、カテゴリ分け (離散化) を使う
- 途中にノイズを足す
- 値を歪める、量子化する、欠損値を作る、といった後処理をかける

なぜ本物のデータではなく、作ったデータを使うのか

TabFM ブログには、次の趣旨が書かれています。

- 実世界の表は、規模も種類も足りず、企業のデータは非公開のことが多い。
- 合成データなら、いくらでも大きく、いくらでも多様に作れる。

論文側の説明もあります。Nature 論文は、テストデータが学習データに混ざる汚染や、著作権の問題を避けられる、と書いています。

v 2につながる課題点や特徴

列ごとに切る GBDT は、役に立たない特徴量に強いが、向きの変化に弱く、滑らかな関数は苦手。

向きに依らない MLP は、役に立たない特徴量に弱い。

TabPFN は、その中間に位置し、事前分布の設計 (単純さ、特徴量の作り方) で、位置が決まる。

ステップ5:3つを1本の話にまとめる

	滑らかさ (B.3.1)	役に立たない特徴量を足す (B.3.2)	回転 (B.3.3)
LightGBM	非滑らか・不規則	ほぼ一定	大きく悪化
MLP	滑らか(低周波)	悪化	影響なし
TabPFN	滑らか(データが多いと複雑にも)	悪化	少し悪化